

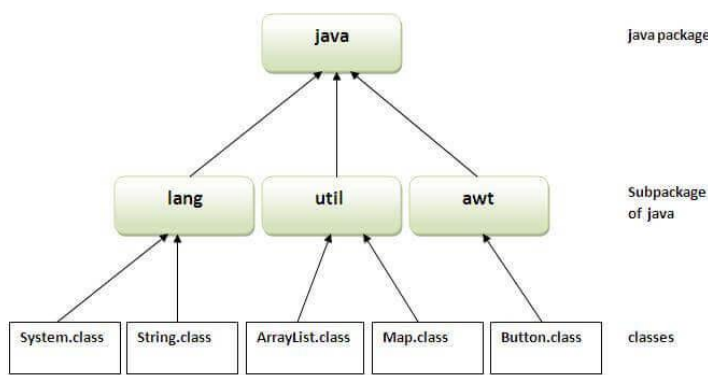
Packages - Access Protection - Importing packages - Exception Handling - Throw and Throws - Thread - Synchronizing - Runnable Interface - Multithreading.

Java Package

A java package is a group of similar types of classes, interfaces and sub-packages. Package in java can be categorized in two form, built-in package and user-defined package. There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc. Here, we will have the detailed learning of creating and using user-defined packages.

Advantage of Java Package

- 1) Java package is used to categorize the classes and interfaces so that they can be easily maintained.
- 2) Java package provides access protection.
- 3) Java package removes naming collision.



Simple example of java package

The package keyword is used to create a package in java.

//save as Simple.java

```

package mypack;
public class Simple{
    public static void main(String args[]){
        System.out.println("Welcome to package");
    }
}
  
```

How to compile java package

If you are not using any IDE, you need to follow the syntax given below:

javac -d directory javafilename

For example

javac -d . Simple.java

The -d switch specifies the destination where to put the generated class file. You can use any directory name like /home (in case of Linux), d:/abc (in case of windows) etc. If you want to keep the package within the same directory, you can use . (dot).

How to run java package program

You need to use fully qualified name e.g. mypack.Simple etc to run the class.

To Compile: javac -d . Simple.java

To Run: java mypack.Simple

Output: Welcome to package

The -d is a switch that tells the compiler where to put the class file i.e. it represents destination. The . represents the current folder.

How to access package from another package?

There are three ways to access the package from outside the package.

1. import package.*;
2. import package.classname;
3. fully qualified name.

1) Using packagename.*

If you use package.* then all the classes and interfaces of this package will be accessible but not subpackages.

The import keyword is used to make the classes and interface of another package accessible to the current package.

Example of package that import the packagename.*

//save by A.java

```
package pack;
public class A{
    public void msg(){System.out.println("Hello");}
}
```

//save by B.java

```
package mypack;
import pack.*;
```

```
class B{
    public static void main(String args[]){
        A obj = new A();
        obj.msg();
    }
}
```

Output: Hello

2) Using packagename.classname

If you import package.classname then only declared class of this package will be accessible.

Example of package by import package.classname

//save by A.java

```
package pack;
public class A{
    public void msg(){System.out.println("Hello");}
}
```

//save by B.java

```
package mypack;
import pack.A;
```

```
class B{
    public static void main(String args[]){
        A obj = new A();
        obj.msg();
    }
}
```

```
}
```

Output:Hello

3) Using fully qualified name

If you use fully qualified name then only declared class of this package will be accessible. Now there is no need to import. But you need to use fully qualified name every time when you are accessing the class or interface.

It is generally used when two packages have same class name e.g. java.util and java.sql packages contain Date class.

Example of package by import fully qualified name

//save by A.java

```
package pack;
```

```
public class A{
```

```
    public void msg(){System.out.println("Hello");}
```

```
}
```

//save by B.java

```
package mypack;
```

```
class B{
```

```
    public static void main(String args[]){
```

```
        pack.A obj = new pack.A();//using fully qualified name
```

```
        obj.msg();
```

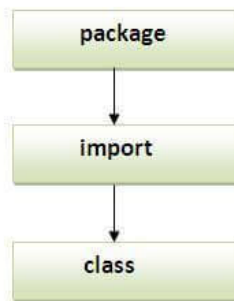
```
    }
```

```
}
```

Output:Hello

If you import a package, all the classes and interface of that package will be imported excluding the classes and interfaces of the subpackages. Hence, you need to import the subpackage as well.

Note: Sequence of the program must be package then import then class.



Access Protection in java packages:

In java, the access modifiers define the accessibility of the class and its members. For example, private members are accessible within the same class members only. Java has four access modifiers, and they are default, private, protected, and public.

In java, the package is a container of classes, sub-classes, interfaces, and sub-packages. The class acts as a container of data and methods. So, the access modifier decides the accessibility of class members across the different packages.

In java, the accessibility of the members of a class or interface depends on its access specifiers. The following table provides information about the visibility of both data members and methods.

Access Specifier \ Accessibility Location	Same Class	Same Package		Other Package	
		Child class	Non-child class	Child class	Non-child class
Public	Yes	Yes	Yes	Yes	Yes
Protected	Yes	Yes	Yes	Yes	No
Default	Yes	Yes	Yes	No	No
Private	Yes	No	No	No	No

- The **public** members can be accessed everywhere.
- The **private** members can be accessed only inside the same class.
- The **protected** members are accessible to every child class (same package or other packages).
- The **default** members are accessible within the same package but not outside the package.

Exception Handling in Java

An exception in java programming is an abnormal situation that is araised during the program execution. In simple words, an exception is a problem that arises at the time of program execution. When an exception occurs, it disrupts the program execution flow. When an exception occurs, the program execution gets terminated, and the system generates an error. We use the exception handling mechanism to avoid abnormal termination of program execution.

Java programming language has a very powerful and efficient exception handling mechanism with a large number of built-in classes to handle most of the exceptions automatically.

Java programming language has the following class hierarchy to support the exception handling mechanism.

Reasons for Exception Occurrence

Several reasons lead to the occurrence of an exception. A few of them are as follows.

When we try to open a file that does not exist may lead to an exception.

When the user enters invalid input data, it may lead to an exception.

When a network connection has lost during the program execution may lead to an exception.

When we try to access the memory beyond the allocated range may lead to an exception.

The physical device problems may also lead to an exception.

Types of Exception

In java, exceptions have categorized into two types, and they are as follows.

Checked Exception - An exception that is checked by the compiler at the time of compilation is called a checked exception.

Unchecked Exception - An exception that can not be caught by the compiler but occurs at the time of program execution is called an unchecked exception.

How exceptions handled in Java?

In java, the exception handling mechanism uses five keywords namely **try**, **catch**, **finally**, **throw**, and **throws**.

Syntax

```
try{
    ...
    code to be tested
    ...
}
catch(ExceptionType object){
    ...
    code for handling the exception
    ...
}
```

The code which is prone to exceptions is placed in the try block. When an exception occurs, that exception thrown is handled by catch block associated with it.

If the type of exception that occurred is listed in a catch block, then the exception is passed to the catch block.

Example: A class not using exception handling

```
class withoutException
{
    public static void main(String args[])
    {

        int num[] = {11, 12,13,14};

        System.out.println(num[5]); System.out.println("
        This is a statement ");

    }
}
```

Output :

```
D:\jdk1.6.0\bin>javac withoutException.java
```

```
D:\jdk1.6.0\bin>java withoutException
```

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 5At
withoutException.main(withoutException.java:9)
```

Example : A class using Exception handling

```
class ExceptionDemo
{
    public static void main(String args[])
    {
        try
        {
```

```

intnum[] = {11,12, 13, 14};
System.out.println(num[5]);
}

catch(ArrayIndexOutOfBoundsException e)
{
System.out.println("Exception thrown : " +e);
}
System.out.println(" This is a statement ");
}
}

```

Output:

D:\java>javac ExceptionDemo.java

D:\java>java ExceptionDemo

Exception thrown :java.lang.ArrayIndexOutOfBoundsException: 5This is a statement

finally statement

A finally statement must be associated with a try statement. It identifies a block of statements that needs to be executed regardless of whether or not an exception occurs within the try block.

Example :

```

classFinallyDemo
{
public static void main(String args[])
{
try
{ int a=10;
int data=25/(a-10);
System.out.println(data);
}
catch(ArithmeticException e)
{
System.out.println(e);
}

finally
{
System.out.println("finally block is always executed");
}

System.out.println("rest of the code...");
}
}

```

OUTPUT:

D:\jdk1.6.0\bin>javac FinallyDemo.java

D:\jdk1.6.0\bin>java FinallyDemo
java.lang.ArithmeticException: / by zero
finally
block is always executed
rest of the code...

Multiple catch clauses

class multicatch

```
{  
    public static void main(String args[])  
    {  
        try  
        {  
            int num[] = {11,12, 13, 14};  
            System.out.println(num[5]);  
            int a = 0;  
            int data=25/a;  
            System.out.println(data);  
        }  
  
        catch(ArithmeticException e)  
        {  
            System.out.println(e);  
        }  
  
        catch(ArrayIndexOutOfBoundsException e1)  
        {  
            System.out.println("Exception thrown : " +e1);  
        }  
        finally  
        {  
            System.out.println("finally block is always executed");  
        }  
        System.out.println("rest of the code...");  
    }  
}
```

Output:

D:\jdk1.6.0\bin>javac multicatch.java

D:\jdk1.6.0\bin>java multicatch
Exception thrown :java.lang.ArrayIndexOutOfBoundsException: 5
finally
block is always executed
rest of the code...

Now inside the try block when exception is thrown, then the type of the exception thrown is compared with the type of exception of each catch block.

If type of exception thrown is matched with the type of exception from catch then the corresponding catch block is executed.

1. At a time only single catch block can be executed. After the execution of catch block control goes to the statement next to the try block.
2. At a time Only single exception can be handled.

throw keyword in Java

The throw keyword is used to throw an exception instance explicitly from a try block to corresponding catch block. That means it is used to transfer the control from try block to corresponding catch block.

The throw keyword must be used inside the try block. When JVM encounters the throw keyword, it stops the execution of try block and jump to the corresponding catch block.

- Using throw keyword only object of Throwable class or its sub classes can be thrown.
- Using throw keyword only one exception can be thrown.
- The throw keyword must followed by an throwable instance.

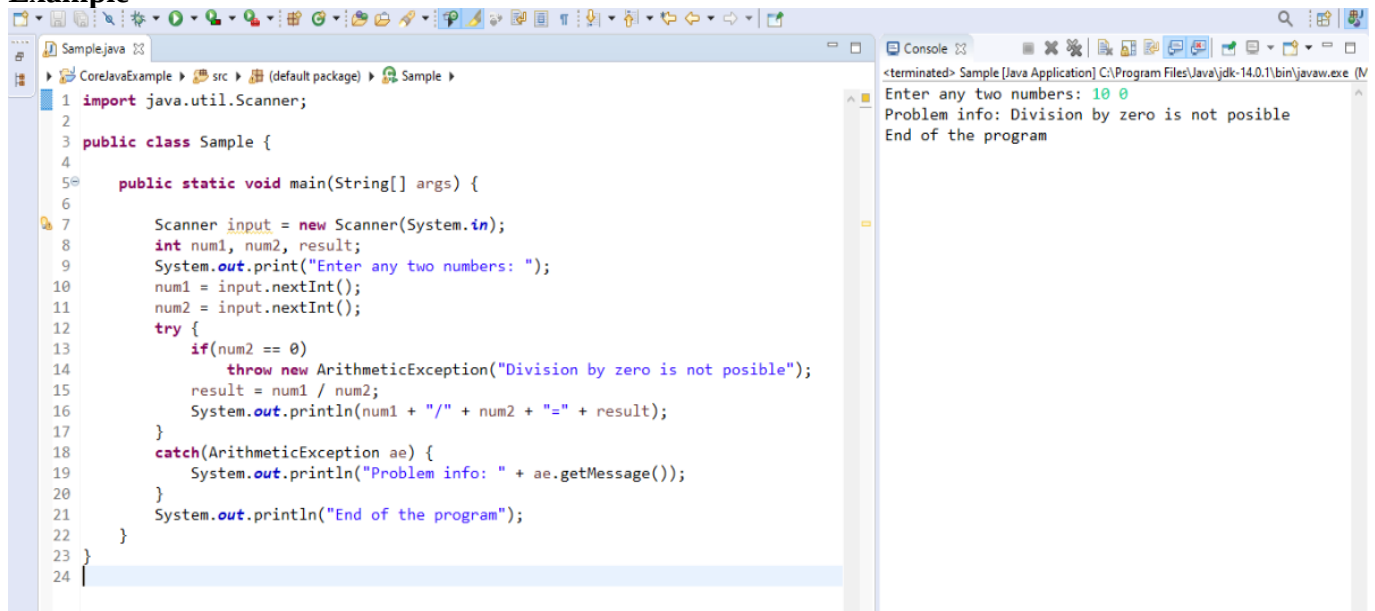
The following is the general syntax for using throw keyword in a try block.

Syntax

throw instance;

Here the instance must be throwable instance and it can be created dynamically using new operator. Let's look at the following example Java code to illustrate throw keyword.

Example



```
1 import java.util.Scanner;
2
3 public class Sample {
4
5     public static void main(String[] args) {
6
7         Scanner input = new Scanner(System.in);
8         int num1, num2, result;
9         System.out.print("Enter any two numbers: ");
10        num1 = input.nextInt();
11        num2 = input.nextInt();
12        try {
13            if(num2 == 0)
14                throw new ArithmeticException("Division by zero is not possible");
15            result = num1 / num2;
16            System.out.println(num1 + "/" + num2 + "=" + result);
17        }
18        catch(ArithmeticException ae) {
19            System.out.println("Problem info: " + ae.getMessage());
20        }
21        System.out.println("End of the program");
22    }
23 }
24 }
```

Console Output:

```
<terminated> Sample [Java Application] C:\Program Files\Java\jdk-14.0.1\bin\javaw.exe (IV
Enter any two numbers: 10 0
Problem info: Division by zero is not possible
End of the program
```

throws keyword in Java

The throws keyword specifies the exceptions that a method can throw to the default handler and does not handle itself. That means when we need a method to throw an exception automatically, we use throws keyword followed by method declaration

- When a method throws an exception, we must put the calling statement of method in try-catch block.

Let's look at the following example Java code to illustrate throws keyword.

```

1 import java.util.Scanner;
2
3 public class ThrowsExample {
4
5
6     int num1, num2, result;
7     Scanner input = new Scanner(System.in);
8
9     void division() throws ArithmeticException {
10         System.out.print("Enter any two numbers: ");
11         num1 = input.nextInt();
12         num2 = input.nextInt();
13         result = num1 / num2;
14         System.out.println(num1 + "/" + num2 + "=" + result);
15     }
16
17     public static void main(String[] args) {
18
19         try {
20             new ThrowsExample().division();
21         }
22         catch(ArithmeticException ae) {
23             System.out.println("Problem info: " + ae.getMessage());
24         }
25         System.out.println("End of the program");
26     }
27 }
28
29

```

Console Output:

```

<terminated> ThrowsExample [Java Application] C:\Program Files\Java\jdk-14.0.1\bin\javaw
Enter any two numbers: 20 0
Problem info: / by zero
End of the program

```

finalize() Method in Java and How to Override it?

The Java **finalize()** method of Object class is a method that the Garbage Collector always calls just before the deletion/destroying the object which is eligible for Garbage Collection to perform clean-up activity. Clean-up activity means closing the resources associated with that object like Database Connection, Network Connection, or we can say resource de-allocation. Remember, it is not a reserved keyword. Once the finalize() method completes immediately, Garbage Collector destroys that object.

Finalization: Just before destroying any object, the garbage collector always calls finalize() method to perform clean-up activities on that object. This process is known as Finalization in Java.

Note: The Garbage collector calls the finalize() method only once on any object.

Syntax:

protected void finalize throws Throwable{}

Since the Object class contains the finalize method hence finalize method is available for every java class since Object is the superclass of all java classes. Since it is available for every java class, Garbage Collector can call the finalize() method on any java object.

Why finalize() method is used?

finalize() method releases system resources before the garbage collector runs for a specific object. JVM allows finalize() to be invoked only once per object.

How to override finalize() method?

The finalize method, which is present in the Object class, has an **empty implementation**. In our class, clean-up activities are there. Then we have to **override this method** to define our clean-up activities.

In order to Override this method, we have to define and call finalize within our code explicitly.
Java

```
// Java code to show the
// overriding of finalize() method
import java.lang.*;

// Defining a class demo since every java class
// is a subclass of predefined Object class
// Therefore demo is a subclass of Object class
public class demo {

    protected void finalize() throws Throwable
    {
        try {

            System.out.println("inside demo's finalize()");
        }
        catch (Throwable e) {

            throw e;
        }
        finally {

            System.out.println("Calling finalize method" + " of the Object class");

            // Calling finalize() of Object class
            super.finalize();
        }
    }

    // Driver code
    public static void main(String[] args) throws Throwable
    {

        // Creating demo's object
        demo d = new demo();

        // Calling finalize of demo
        d.finalize();
    }
}
```

Output:

inside demo's finalize()
Calling finalize method of the Object class

D:\jdk1.6.0\bin>javac withoutException.java

D:\jdk1.6.0\bin>java withoutException

Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 5At
withoutException.main(withoutException.java:9)

Example : A class using Exception handling

```
class ExceptionDemo
{
    public static void main(String args[])
    {
        try
        {
            intnum[] = {11,12, 13, 14};
            System.out.println(num[5]);
        }

        catch(ArrayIndexOutOfBoundsException e)
        {
            System.out.println("Exception thrown : " +e);
        }
        System.out.println(" This is a statement “);
    }
}
```

Output:

D:\java>javac ExceptionDemo.java

D:\java>java ExceptionDemo

Exception thrown :java.lang.ArrayIndexOutOfBoundsException: 5This is a
statement

finally statement

A finally statement must be associated with a try statement. It identifies a block of statements that needs to be executed regardless of whether or not an exception occurs within the try block.

Example :

```
class FinallyDemo
{
    public static void main(String args[])
    {
        try
        { int a=10;
```

```

int data=25/(a-10);
System.out.println(data);
    }
catch(ArithmeticException e)
    {
System.out.println(e);
    }

finally
    {
System.out.println("finally block is always executed");
    }

System.out.println("rest of the code...");
    }
}

```

OUTPUT:

D:\jdk1.6.0\bin>javac FinallyDemo.java

D:\jdk1.6.0\bin>java FinallyDemo java.lang.ArithmeticException:
/ by zero
finally block is always executed
rest of the code...

Multiple catch clauses

```

class multicatch
{
public static void main(String args[])
{
    try
    {
        int num[] = {11,12, 13, 14};
        System.out.println(num[5]);int a =
        0;
        int data=25/a;
        System.out.println(data);
    }

    catch(ArithmeticException e)
    {
        System.out.println(e);
    }

    catch(ArrayIndexOutOfBoundsException e1)
    {
        System.out.println("Exception thrown : " +e1);
    }
}
}

```

```

        }
        finally
        {
            System.out.println("finally block is always executed");
        }
    }
    System.out.println("rest of the code...");
}

```

Output:

D:\jdk1.6.0\bin>javac multicatch.java

D:\jdk1.6.0\bin>java multicatch
 Exception thrown :java.lang.ArrayIndexOutOfBoundsException:
 finally block is always executed
 rest of the code...

Now inside the try block when exception is thrown, then the type of the exception thrown is compared with the type of exception of each catch block.

If type of exception thrown is matched with the type of exception from catch then the corresponding catch block is executed.

1. At a time only single catch block can be executed. After the execution of catch block control goes to the statement next to the try block.
2. At a time Only single exception can be handled.

Java Threads

Threads allows a program to operate more efficiently by doing multiple things at the same time. Threads can be used to perform complicated tasks in the background without interrupting the main program.

Creating a Thread

There are two ways to create a thread.

It can be created by extending the Thread class and overriding its run() method:

Extend Syntax

```

public class Main extends Thread {
    public void run() {
        System.out.println("This code is running in a thread");
    }
}

```

Another way to create a thread is to implement the Runnable interface:

Implement Syntax

```

public class Main implements Runnable {
    public void run() {
        System.out.println("This code is running in a thread");
    }
}

```

Running Threads

If the class extends the Thread class, the thread can be run by creating an instance of the class and call its start() method:

Extend Example

```

public class Main extends Thread {
    public static void main(String[] args) {
        Main thread = new Main();
        thread.start();
        System.out.println("This code is outside of the thread");
    }
}

```

```

public void run() {
    System.out.println("This code is running in a thread");
}
}

```

This code is outside of the thread
This code is running in a thread

Implement Example

```

public class Main implements Runnable {
    public static void main(String[] args) {
        Main obj = new Main();
        Thread thread = new Thread(obj);
        thread.start();
        System.out.println("This code is outside of the thread");
    }
    public void run() {
        System.out.println("This code is running in a thread");
    }
}

```

This code is outside of the thread
This code is running in a thread

Differences between "extending" and "implementing" Threads

The major difference is that when a class extends the Thread class, you cannot extend any other class, but by implementing the Runnable interface, it is possible to extend from another class as well, like: class MyClass extends OtherClass implements Runnable.

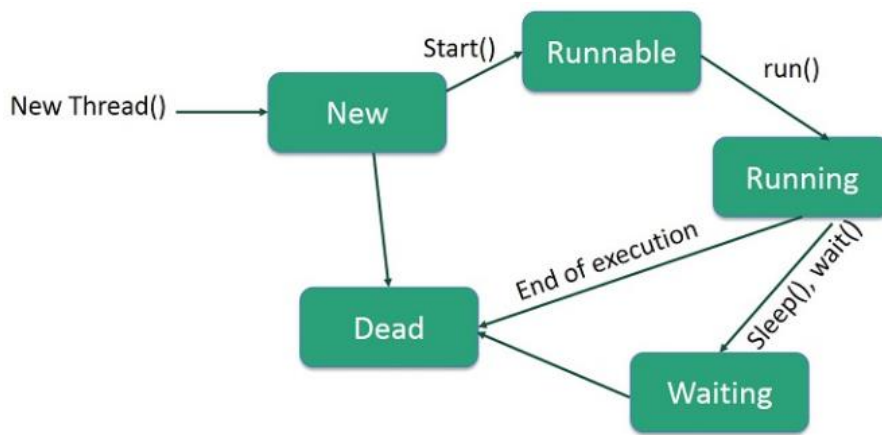
Commonly used methods of Thread class:

public void run(): is used to perform action for a thread.
public void start(): starts the execution of the thread.JVM calls the run() method on the thread.
public void sleep(long miliseconds): Causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds..
public int getPriority(): returns the priority of the thread.
public int setPriority(int priority): changes the priority of the thread.
public String getName(): returns the name of the thread.
public void setName(String name): changes the name of the thread.
public Thread currentThread(): returns the reference of currently executing thread.
public int getId(): returns the id of the thread.
public Thread.State getState(): returns the state of the thread.
public boolean isAlive(): tests if the thread is alive.
public void yield(): causes the currently executing thread object to temporarily pause and allow other threads to execute.
public void suspend(): is used to suspend the thread(deprecated).
public void resume(): is used to resume the suspended thread(deprecated).
public void stop(): is used to stop the thread(deprecated).
public void interrupt(): interrupts the thread.
public boolean isInterrupted(): tests if the thread has been interrupted.
public static boolean interrupted(): tests if the current thread has been interrupted.

Life Cycle of a Thread in Java

The life cycle of a thread in Java refers to the various states of a thread goes through. For example, a thread is born, started, runs, and then dies. Thread class defines the life cycle and various states of a thread.

Flow Chart of Java Thread Life Cycle



States of a Thread Life Cycle in Java

Following are the stages of the life cycle –

New – A new thread begins its life cycle in the new state. It remains in this state until the program starts the thread. It is also referred to as a born thread.

Runnable – After a newly born thread is started, the thread becomes runnable. A thread in this state is considered to be executing its task.

Waiting – Sometimes, a thread transitions to the waiting state while the thread waits for another thread to perform a task. A thread transitions back to the runnable state only when another thread signals the waiting thread to continue executing.

Timed Waiting – A runnable thread can enter the timed waiting state for a specified interval of time. A thread in this state transitions back to the runnable state when that time interval expires or when the event it is waiting for occurs.

Terminated (Dead) – A runnable thread enters the terminated state when it completes its task or otherwise terminates.

Thread.sleep() in Java with Examples

The Java Thread class provides the two variant of the sleep() method. First one accepts only an arguments, whereas the other variant accepts two arguments. The method sleep() is being used to halt the working of a thread for a given amount of time. The time up to which the thread remains in the sleeping state is known as the sleeping time of the thread. After the sleeping time is over, the thread starts its execution from where it has left

The sleep() Method Syntax:

Following are the syntax of the sleep() method.

public static void sleep(long mls) throws InterruptedException

public static void sleep(long mls, int n) throws InterruptedException

Parameters:

The following are the parameters used in the sleep() method.

mls: The time in milliseconds is represented by the parameter mls. The duration for which the thread will sleep is given by the method sleep().

n: It shows the additional time up to which the programmer or developer wants the thread to be in the sleeping state. The range of n is from 0 to 999999.

The method does not return anything.

Important Points to Remember About the Sleep() Method

Whenever the Thread.sleep() methods execute, it always halts the execution of the current thread.

Whenever another thread does interruption while the current thread is already in the sleep mode, then the InterruptedException is thrown.

If the system that is executing the threads is busy, then the actual sleeping time of the thread is generally more as compared to the time passed in arguments. However, if the system executing the sleep() method has less load, then the actual sleeping time of the thread is almost equal to the time passed in the argument.

```
// important import statements
import java.lang.Thread;
import java.io.*;

public class TestSleepMethod2
{
    // main method
    public static void main(String args[])
    {

        try {
            for (int j = 0; j < 5; j++)
            {

                // The main thread sleeps for the 1000 milliseconds, which is 1 sec
                // whenever the loop runs
                Thread.sleep(1000);

                // displaying the value of the variable
                System.out.println(j);
            }
        } catch (Exception expn)
        {
            // catching the exception
            System.out.println(expn);
        }
    }
}

Output:0
1
2
3
4
```

Multithreading in Java

Multithreading is a Java feature that allows concurrent execution of two or more parts of a program for maximum utilization of CPU. Each part of such program is called a thread. So, threads are light-weight processes within a process.

Threads can be created by using two mechanisms :

Extending the Thread class

Implementing the Runnable Interface

Thread creation by extending the Thread class

We create a class that extends the **java.lang.Thread** class. This class overrides the run() method available in the Thread class. A thread begins its life inside run() method. We create an object of our new class and call start() method to start the execution of a thread. Start() invokes the run() method on the Thread object.

// Java code for thread creation by extending

// the Thread class

```
class MultithreadingDemo extends Thread {
```

```
    public void run()
```

```
    {
```

```
    {
```

```
        try {
```

```
            // Displaying the thread that is running
```

```
            System.out.println(
```



```

        "Thread " + Thread.currentThread().getId()
        + " is running");
    }
    catch (Exception e) {
        // Throwing an exception
        System.out.println("Exception is caught");
    }
}
}

```

// Main Class

```

public class Multithread {
    public static void main(String[] args)
    {
        int n = 8; // Number of threads
        for (int i = 0; i < n; i++) {
            MultithreadingDemo object
                = new MultithreadingDemo();
            object.start();
        }
    }
}

```

Output

```

Thread 15 is running
Thread 14 is running
Thread 16 is running
Thread 12 is running
Thread 11 is running
Thread 13 is running
Thread 18 is running
Thread 17 is running

```

Thread creation by implementing the Runnable Interface

We create a new class which implements java.lang.Runnable interface and override run() method. Then we instantiate a Thread object and call start() method on this object.

// Java code for thread creation by implementing

// the Runnable Interface

```

class MultithreadingDemo implements Runnable {
    public void run()
    {
        try {
            // Displaying the thread that is running
            System.out.println(
                "Thread " + Thread.currentThread().getId()
                + " is running");
        }
        catch (Exception e) {
            // Throwing an exception
            System.out.println("Exception is caught");
        }
    }
}

```

17

// Main Class

```

class Multithread {

```

```

public static void main(String[] args)
{
    int n = 8; // Number of threads
    for (int i = 0; i < n; i++) {
        Thread object
            = new Thread(new MultithreadingDemo());
        object.start();
    }
}
}
Thread 13 is running
Thread 11 is running
Thread 12 is running
Thread 15 is running
Thread 14 is running
Thread 18 is running
Thread 17 is running
Thread 16 is running

```

Thread Class vs Runnable Interface

If we extend the Thread class, our class cannot extend any other class because Java doesn't support multiple inheritance. But, if we implement the Runnable interface, our class can still extend other base classes.

We can achieve basic functionality of a thread by extending Thread class because it provides some inbuilt methods like yield(), interrupt() etc. that are not available in Runnable interface.

Using runnable will give you an object that can be shared amongst multiple threads.

Java Thread Priority in Multithreading

Priorities in threads is a concept where each thread is having a priority which in layman's language one can say every object is having priority here which is represented by numbers ranging from 1 to 10.

The default priority is set to 5 as excepted.

Minimum priority is set to 1.

Maximum priority is set to 10.

Here 3 constants are defined in it namely as follows:

```

public static int NORM_PRIORITY
public static int MIN_PRIORITY
public static int MAX_PRIORITY

```

In this example, we've created a ThreadDemo class which extends Thread class. We've created three threads. As we're not setting any priority, each thread has a normal priority. In run() method, we're printing the priorities and in output, threads are executing in any order.

```

class ThreadDemo extends Thread {
    ThreadDemo() {
    }
    public void run() {
        System.out.println("Thread Name: " + Thread.currentThread().getName()
            + ", Thread Priority: " + Thread.currentThread().getPriority());
        for(int i = 4; i > 0; i--) {
            System.out.println("Thread: " + Thread.currentThread().getName() + ", " + i);
        }
        try {
            Thread.sleep(50);
        } catch (InterruptedException e) {

        }
    }
}
// TODO Auto-generated catch block

```

```

        e.printStackTrace();
    }
}
public void start () {
    super.start();
}
}
public class TestThread {
    public static void main(String args[]) {
        ThreadDemo thread1 = new ThreadDemo();
        ThreadDemo thread2 = new ThreadDemo();
        ThreadDemo thread3 = new ThreadDemo();
        thread1.start();
        thread2.start();
        thread3.start();
    }
}

```

Thread Name: Thread-1, Thread Priority: 5
 Thread Name: Thread-2, Thread Priority: 5
 Thread Name: Thread-0, Thread Priority: 5
 Thread: Thread-2, 4
 Thread: Thread-1, 4
 Thread: Thread-1, 3
 Thread: Thread-2, 3
 Thread: Thread-0, 4
 Thread: Thread-2, 2
 Thread: Thread-1, 2
 Thread: Thread-2, 1
 Thread: Thread-0, 3
 Thread: Thread-1, 1
 Thread: Thread-0, 2
 Thread: Thread-0, 1

Synchronization in Java

Multi-threaded programs may often come to a situation where multiple threads try to access the same resources and finally produce erroneous and unforeseen results.

Why use Java Synchronization?

Java Synchronization is used to make sure by some synchronization method that only one thread can access the resource at a given point in time.

Multithreading Example without Thread Synchronization

```

class PrintDemo {
    public void printCount() {
        try {
            for(int i = 5; i > 0; i--) {
                System.out.println("Counter --- " + i);
            }
        } catch (Exception e) {
            System.out.println("Thread interrupted.");
        }
    }
}

```

```

class ThreadDemo extends Thread {
    private Thread t;
    private String threadName;
    PrintDemo PD;
}

```

```

ThreadDemo( String name, PrintDemo pd) {
    threadName = name;
    PD = pd;
}

public void run() {
    PD.printCount();
    System.out.println("Thread " + threadName + " exiting.");
}

public void start () {
    System.out.println("Starting " + threadName );
    if (t == null) {
        t = new Thread (this, threadName);
        t.start ();
    }
}
}

public class TestThread {
    public static void main(String args[]) {

        PrintDemo PD = new PrintDemo();

        ThreadDemo T1 = new ThreadDemo( "Thread - 1 ", PD );
        ThreadDemo T2 = new ThreadDemo( "Thread - 2 ", PD );

        T1.start();
        T2.start();

        // wait for threads to end
        try {
            T1.join();
            T2.join();
        } catch ( Exception e) {
            System.out.println("Interrupted");
        }
    }
}

```

Output

```

Starting Thread - 1
Starting Thread - 2
Counter --- 5
Counter --- 4
Counter --- 3
Counter --- 5
Counter --- 2
Counter --- 1
Counter --- 4
Thread Thread - 1 exiting.
Counter --- 3
Counter --- 2
Counter --- 1
Thread Thread - 2 exiting.

```

20

Multithreading Example with Thread Synchronization

Here is the same example which prints counter value in sequence and every time we run it, it produces the same result.

```

class PrintDemo {

```

```

public void printCount() {
    try {
        for(int i = 5; i > 0; i--) {
            System.out.println("Counter --- " + i );
        }
    } catch (Exception e) {
        System.out.println("Thread interrupted.");
    }
}

}

class ThreadDemo extends Thread {
    private Thread t;
    private String threadName;
    PrintDemo PD;

    ThreadDemo( String name, PrintDemo pd) {
        threadName = name;
        PD = pd;
    }

    public void run() {
        synchronized(PD) {
            PD.printCount();
        }
        System.out.println("Thread " + threadName + " exiting.");
    }

    public void start () {
        System.out.println("Starting " + threadName );
        if (t == null) {
            t = new Thread (this, threadName);
            t.start ();
        }
    }
}

public static void main(String args[]) {
    PrintDemo PD = new PrintDemo();

    ThreadDemo T1 = new ThreadDemo( "Thread - 1 ", PD );
    ThreadDemo T2 = new ThreadDemo( "Thread - 2 ", PD );

    T1.start();
    T2.start();

    // wait for threads to end
    try {
        T1.join();
        T2.join();
    } catch ( Exception e) {
        System.out.println("Interrupted");
    }
}
}

```

Output

```

Starting Thread - 1
Starting Thread - 2
Counter --- 5
Counter --- 4

```

Counter --- 3
Counter --- 2
Counter --- 1
Thread Thread - 1 exiting.
Counter --- 5
Counter --- 4
Counter --- 3
Counter --- 2
Counter --- 1
Thread Thread - 2 exiting.